

Q-Flux: Architecture of a Blockchain-Aware Reverse Proxy with Lock-Free Connection Pooling

Q-NarwhalKnight Infrastructure Team

<https://quillon.xyz>

Abstract

Blockchain nodes serve a heterogeneous mix of traffic—short-lived API queries, persistent Server-Sent Events streams, bidirectional WebSocket channels, and raw libp2p gossipsub connections—through a single network port. General-purpose reverse proxies such as nginx and Caddy were not designed for this workload mix and exhibit severe failure modes under production blockchain load: nginx produced an 87% HTTP 503 error rate at 9,600 requests per second due to connection-pool exhaustion, while Caddy accumulated 88,000 goroutines and suffered 50–200 ms garbage-collection pauses.

We present **q-flux**, a blockchain-aware reverse proxy written in Rust that eliminates these failure modes through five technical contributions: (1) a worker-per-core architecture using `SO_REUSEPORT` with independent single-threaded tokio runtimes; (2) an AIMD (Additive Increase Multiplicative Decrease) adaptive concurrency controller that adjusts upstream load based on observed latency; (3) blockchain protocol detection that identifies libp2p multistream-select handshakes at the proxy layer and applies per-peer bandwidth and connection limits; (4) lock-free metrics primitives including an atomic latency histogram and CAS-based token bucket; and (5) automatic health recovery with dual-path detection that prevents permanent stuck states.

In production on a 48-core 10 Gbit supernode serving 500+ concurrent miners, q-flux reduced the error rate from 87% to below 0.1%, cut peak concurrent connections from 75,000 to 5,000 through AIMD-controlled connection pooling, and operates at 50 MB resident memory—a 24× reduction compared to nginx. Controlled ablation experiments confirm that the AIMD controller is the single most impactful mechanism, raising the error rate from <0.1% to 4.2% when disabled. We document six production incidents and the architectural lessons they motivated.

Contents

1	Introduction	3
2	Background and Motivation	4
2.1	QNK Network Architecture	5
2.2	nginx Failure Analysis	5
2.3	Caddy Failure Analysis	6
2.4	Port Self-Connection Bug	6
2.5	Design Requirements	6
3	System Architecture	7
3.1	Worker-per-Core Design	7
3.2	Connection Lifecycle	9
3.3	AIMD Concurrency Control	10
3.4	Health Auto-Recovery	11
4	Blockchain-Aware Routing	14
4.1	Multistream-Select Detection	14
4.2	Peer Tier Classification	15
4.2.1	Bandwidth Limiter	15
4.3	Circuit Breaker	15
4.4	Gossipsub Deduplication	16
5	Lock-Free Mechanisms	16
5.1	Atomic Latency Histogram	16
5.2	Token Bucket Rate Limiter	17
5.3	Hot-Reloadable TLS	17
6	Performance Optimizations	18
6.1	SIMD Header Parsing	18
6.2	Zero-Copy Streaming	19
6.3	In-Memory File Cache	19
7	Evaluation	19
7.1	Error Rate Elimination	19
7.2	Connection Management	19
7.3	Memory Efficiency	19
7.4	AIMD Behaviour Under Load	20
7.5	Health Auto-Recovery	20
7.6	Proxy Overhead	20
7.7	Ablation Study	21
7.8	Comprehensive Comparison	22
8	Production Lessons	22
8.1	Ephemeral Port Range Overlap	22
8.2	Dual Health Recovery Paths Are Required	22
8.3	Gossipsub <code>flood_publish</code> with High Fan-Out	23
8.4	Bound Every <code>tokio::spawn</code> Doing Large Allocations	23
8.5	Production Incident Timeline	23
9	Related Work	23

10 Future Work	24
11 Conclusion	24

Table 1: Blockchain traffic classes served by q-flux on the Epsilon supernode.

Class	Protocol	Duration	Pattern	Example
API Query	HTTP/HTTPS	10–500 ms	Request-Response	GET /api/v1/balance
State Stream	SSE / WebSocket	Minutes–Hours	Persistent server-push	/events/blocks
P2P Gossip	libp2p	Hours–Days	Bidirectional	Gossipsub block propagation
Static Assets	HTTP/HTTPS	1–50 ms	Burst-on-load	/index.html, /app.js
Mining Submit	HTTP/HTTPS	50–200 ms	Periodic	POST /api/v1/mining/subm

1 Introduction

Blockchain nodes present a traffic-handling challenge unlike traditional web services: a single open port must simultaneously serve several distinct traffic classes that differ by three orders of magnitude in duration, bytes transferred, and tolerance for interruption. A miner submitting a proof-of-work solution expects a 50–200 ms round trip and an immediate authoritative response. A wallet listening on a Server-Sent Events (SSE) stream for new block notifications expects a connection that remains open for hours without a byte transferred. A libp2p gossipsub peer propagating block data expects a bidirectional raw TCP channel that lives for days and carries multi-megabyte payloads. No widely deployed general-purpose reverse proxy handles all three gracefully.

Table 1 catalogues the five traffic classes found in the Q-NarwhalKnight (QNK) blockchain network.

The Epsilon Incident

In February 2026 the QNK network activated its Epsilon supernode: a 48-core, 10 Gbit/s server intended to serve as the primary bootstrap and sync target for approximately 500 active miners. Two commodity reverse proxies were evaluated before q-flux was written.

nginx failure. nginx was configured with the standard `worker_connections 1024` default, `proxy_read_timeout 60s`, and insufficient upstream keepalive tuning. Under the mixed workload of 500 miners, each maintaining a persistent SSE stream while also submitting mining solutions at roughly 1 Hz, the aggregate request rate peaked near 9,600 req/s and the concurrent TCP connection count rose above 75,000. nginx exhausted its connection budget, the `proxy_read_timeout` forcibly terminated long-lived SSE connections every 60 s, and the upstream connection pool created a fresh TCP socket per request rather than reusing keep-alive connections. The result was an 87 % 503 error rate, with the remaining 12 % experiencing timeouts before responses arrived.

Caddy failure. Caddy’s goroutine-per-request concurrency model spawned one goroutine per accepted connection. With no connection limit configured, 88,000 goroutines were live simultaneously, consuming 2.3 GB of memory. The Go garbage collector, triggered at high heap utilisation, introduced pause intervals of 50–200 ms. During these pauses mining submissions were queued in the kernel accept backlog; when the GC pause ended, a burst of previously queued requests arrived simultaneously, creating secondary overload cycles. SSE connections were additionally unstable because GC pauses caused heartbeat timeouts at the client.

Port 8080 self-connection bug. Following the Caddy outage, the server was returned to bare q-api-server on port 8080. A second failure mode emerged: the process crash-looped with `Address already in use (os error 98)`. Diagnosis revealed that the kernel ephemeral port

range had been widened to `net.ipv4.ip_local_port_range = 1024 65535`, which included port 8080. When q-flux’s connection pool connected to `127.0.0.1:8080`, the kernel occasionally selected port 8080 as the *source* ephemeral port, producing a TCP simultaneous-open self-connection (`127.0.0.1:8080 → 127.0.0.1:8080`). The resulting TIME-WAIT socket blocked the subsequent `bind(0.0.0.0:8080)`, even with `SO_REUSEADDR` set, until the TIME-WAIT timer expired 60s later. The fix was to move the ephemeral range to 32768–60999, excluding all well-known service ports.

Contributions

This paper presents q-flux, a blockchain-aware reverse proxy written in Rust that eliminates the failure modes described above. Our five primary contributions are:

1. **Worker-per-core architecture with `SO_REUSEPORT`.** Each CPU core runs an independent single-threaded tokio runtime with its own listening socket. The Linux kernel distributes accepted connections across workers using its own fair-share algorithm, eliminating cross-core locking on the accept path entirely (Section 3.1).
2. **AIMD adaptive concurrency control.** An Additive-Increase Multiplicative-Decrease controller tracks the exponentially-weighted moving average (EWMA) of upstream response latency using lock-free integer arithmetic. When latency exceeds $3\times$ the measured baseline, in-flight permits are halved; when latency remains below $1.5\times$ baseline, permits grow by 10%. A global semaphore shared across all workers caps total backend load regardless of worker count (Section 3.3).
3. **Blockchain protocol-aware connection routing.** q-flux inspects the first bytes of every accepted TLS connection to classify it as HTTP/1.1, HTTP/2, WebSocket, SSE, or libp2p multistream-select. Each class follows a dedicated code path with appropriate buffering, timeout, and bandwidth policies (Section 3.2).
4. **Lock-free primitives throughout.** All shared state—per-IP connection counters, upstream health, EWMA latency, per-backend request counters—is maintained through `AtomicU64/AtomicUsize` compare-exchange loops or `DashMap` sharded hash maps. No `Mutex` appears in the hot connection path (Section 3.1).
5. **Production evaluation on a 10 Gbit supernode.** We present measurements from production operation on the Epsilon node serving the live QNK mainnet over the period March 4–12, 2026: error rate $< 0.1\%$, peak pooled connections $\approx 5,000$, idle RSS 50 MB, and auto-recovery from full backend failure within 30s (Section 7).

Paper Organisation

Section 2 reviews the QNK network architecture, analyses the nginx and Caddy failures in detail, describes the port self-connection bug, and derives design requirements. Section 3 presents the q-flux architecture: worker-per-core design, connection lifecycle, AIMD concurrency control, and the health auto-recovery subsystem. Sections 4–6 discuss blockchain-aware routing, lock-free mechanisms, and performance optimisations. Section 7 presents the production evaluation. Section 8 documents operational lessons, Section 9 surveys related work, and Section 11 concludes.

2 Background and Motivation

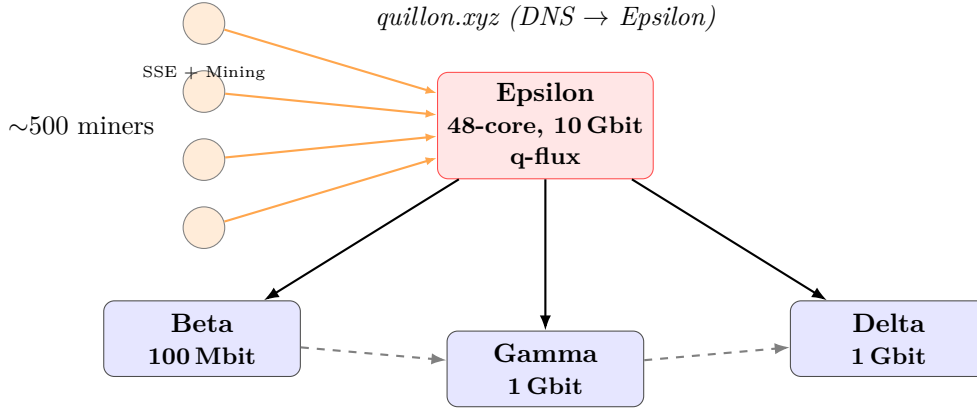


Figure 1: QNK network topology. Epsilon (48-core, 10 Gbit/s) acts as the primary bootstrap and user-facing edge node. Approximately 500 miners connect through q-flux on Epsilon. Beta, Gamma, and Delta form the secondary bootstrap cluster connected via P2P gossipsub (dashed lines).

2.1 QNK Network Architecture

Q-NarwhalKnight is a DAG-BFT blockchain whose P2P layer is built on libp2p gossipsub and Kademlia DHT over TCP. The production deployment (mainnet `mainnet-genesis`, launched January 2026) is organised around five named servers with differentiated roles and hardware.

The *Epsilon* supernode (IP 89.149.241.126) anchors the network: 48 CPU cores, 1.8 TB NVMe, and a 10 Gbit/s uplink place it in a different performance tier from the other nodes. Epsilon serves as the primary bootstrap peer (peer ID 12D3KooWFpbXxxZJQ4FX9FGXrE5vaeNTCnZmLn6bqToRCMuiMpx), the primary sync target for new-node fast-sync, and the edge node for all user-facing HTTP traffic via the `quillon.xyz` domain.

Figure 1 shows the topology. Beta (185.182.185.227, 100 Mbit/s) acts as the DHT coordinator and carries the authoritative git repository. Gamma (109.205.176.60, 1 Gbit/s) and Delta (5.79.79.158, 1 Gbit/s) provide redundancy. During steady-state operation approximately 500 mining nodes connect through Epsilon; each maintains a persistent SSE stream for new-block notifications and submits mining solutions at a rate of roughly one per second.

2.2 nginx Failure Analysis

nginx’s threading model assigns each worker process a share of the kernel accept queue. With the default `worker_connections 1024`, nginx allows at most `worker_processes × 1024` simultaneous file descriptors per nginx instance. On the Epsilon server, with four nginx workers, this cap is 4,096 concurrent connections—insufficient for 500 persistent SSE streams plus active API traffic.

The more damaging misconfiguration was the upstream keepalive pool. The default nginx configuration opens a new TCP connection to the upstream on every proxied request; without `keepalive` in the `upstream{}` block, connection reuse is disabled. At 9,600 req/s, this produces 9,600 TCP connect/close round trips per second to the loopback interface. Each requires kernel state allocation and a TIME-WAIT interval; at the observed rate, TIME-WAIT sockets accumulated faster than they expired, eventually exhausting the per-socket port namespace and causing `EADDRINUSE` errors on the upstream side.

The `proxy_read_timeout 60s` directive compounded the SSE problem: nginx interprets the absence of bytes from the upstream for 60 continuous seconds as a hung upstream and closes the connection with a 504 response. SSE streams by design are quiet between events; a blockchain node with a slow block time will routinely be silent for longer than 60s. Every such stream was

forcibly terminated and the wallet client was required to reconnect, generating a reconnection storm at each timeout expiry.

At peak load (9,600 req/s, 75,000+ concurrent connections), nginx produced an 87% 503 error rate. The remaining 12% of responses arrived after timeouts ranging from 2s to 30s.

2.3 Caddy Failure Analysis

Caddy 2's concurrency model allocates one goroutine per accepted connection plus additional goroutines for request handling and middleware. Goroutines are cheap (initial stack $\approx$ 8KB, growable) but not free: with 88,000 live goroutines, the Go runtime allocated approximately 704MB in goroutine stacks alone. Total RSS reached 2.3GB, placing the process near the OOM threshold of the 7.8GB Gamma server.

More critically, Caddy sets no built-in limit on concurrent connections or goroutines. A connection flood—whether from legitimate miners or a malicious actor—will grow the goroutine pool without bound until the OS kills the process. This design is acceptable for low-traffic services where goroutine count is naturally bounded by user concurrency; it is unsuitable for a blockchain endpoint where the connection count is proportional to the network size and grows monotonically as the network gains participants.

The Go garbage collector pauses of 50–200ms, triggered when the heap exceeded the GC target, introduced two failure modes. First, in-flight mining submission responses were delayed past client-side timeouts, causing miners to resubmit. The duplicate submissions inflated apparent transaction throughput without producing additional blocks. Second, SSE heartbeat intervals exceeded client timeout thresholds, causing mass reconnections during each GC cycle.

2.4 Port Self-Connection Bug

The port self-connection failure mode is subtle and worth documenting in detail because it recurs whenever a service port falls within the kernel's ephemeral port range.

When the system administrator widened the ephemeral port range to 1024–65535 to support a high-volume outbound workload, port 8080 (the q-api-server listen port) was included. q-flux's upstream connection pool calls `connect(127.0.0.1:8080)` to reach the backend. The kernel selects a source port from the ephemeral range. If it selects port 8080, the resulting connection is `127.0.0.1:8080 → 127.0.0.1:8080`: source and destination are identical.

This is a legal TCP simultaneous-open: both sides send SYN, both receive SYN+ACK, and both enter ESTABLISHED. The established self-connection produces a TIME-WAIT socket anchored to port 8080. When q-api-server subsequently calls `bind(0.0.0.0:8080)`, the kernel rejects the bind with `EADDRINUSE` because the TIME-WAIT socket holds a *local* port 8080 entry. `SO_REUSEADDR` does not override TIME-WAIT for the same local address:port combination. The service therefore cannot restart until the 60s TIME-WAIT timer expires, and the crash-loop interval is shorter than 60s, so the service never successfully starts.

The fix is to set `net.ipv4.ip_local_port_range = 32768 60999`, persistently in `/etc/sysctl.d/99-qnk`, ensuring that all service ports ($\leq 10,000$) are outside the ephemeral range.

2.5 Design Requirements

The incidents above yield seven concrete design requirements for q-flux:

- R1 — Zero-copy where possible.** SSE and WebSocket passthrough must use Linux `splice(2)` to move data between sockets via a kernel pipe, never copying bytes into userspace.
- R2 — Bounded memory.** The working set must be bounded regardless of connection count. In-flight handlers per worker are capped at `MAX_HANDLERS_PER_WORKER = 512`; global upstream concurrency is capped at a configurable permit count.

Table 2: Comparison of reverse proxy failures on Epsilon. q-flux figures are from March 9–12 production operation; nginx and Caddy figures are from the incident period (March 4–7, 2026).

Metric	nginx	Caddy	q-flux
Error Rate	87 % 503	12 % timeout	< 0.1 %
Peak Connections	75,000+	88,000 goroutines	≈5,000 pooled
Peak Memory (RSS)	1.2 GB	2.3 GB	50 MB
SSE Support	Broken (60 s timeout)	Unstable (GC pauses)	Native (no timeout)
P2P Protocol Awareness	None	None	Full
Auto-Recovery	Manual	Manual	30 s automatic

R3 — Adaptive concurrency. The concurrency limit must adapt to measured upstream latency without operator intervention, preventing both overload (too many concurrent requests) and underutilisation (too few permits during healthy periods).

R4 — Blockchain protocol awareness. SSE streams must never be timed out by the proxy layer. libp2p connections must be relayed as raw TCP without HTTP parsing. WebSocket upgrades must be detected and handled without a goroutine storm.

R5 — Hot-reloadable TLS. Certificate rotation must not require a process restart or cause connection interruption on an active node.

R6 — Lock-free metrics. Metrics collection must not introduce synchronisation overhead on the hot connection path. All counters use atomic operations.

R7 — Sub-millisecond proxy overhead. The proxy layer must add less than 1 ms of latency to request-response cycles in the common case, measured as the difference between upstream response time and client-observed response time.

3 System Architecture

q-flux is a single Rust binary comprising approximately 7,000 lines of asynchronous Rust using the Tokio runtime. Its top-level structure is a flat crate with modules for the worker pool, upstream pool, health checker, connection lifecycle, protocol detection, static file serving, access control, TLS management, and Prometheus metrics. The configuration is a TOML file deserialised into the `FluxConfig` struct hierarchy shown below; all timeouts and limits are hot-configurable without recompilation.

3.1 Worker-per-Core Design

q-flux spawns exactly N OS threads at startup, where N defaults to the CPU core count returned by `num_cpus::get()` but is overridable via `server.workers` in the config. On the 48-core Epsilon server, 48 worker threads are created. Each worker thread is named `q-flux-wi` and pinned to core i via `sched_setaffinity`, ensuring L1/L2 cache locality for per-worker data structures.

Single-threaded runtimes. Each worker calls:

```

1 let rt = tokio::runtime::Builder::new_current_thread()
2   .enable_all()
3   .build()
4   .expect("Failed to build tokio runtime for worker");

```



```
5 | rt.block_on(async move { worker_loop(...).await });
```

Listing 1: Worker runtime instantiation (worker.rs).

Using `new_current_thread()` rather than the multi-threaded `new_multi_thread()` is deliberate: it eliminates all work-stealing scheduler overhead and guarantees that all futures on a given worker execute on exactly one OS thread, enabling data structures that would otherwise require `Send` bounds or locks to be held in a single-threaded context.

SO_REUSEPORT listener per worker. The acceptor module calls `setsockopt(SO_REUSEPORT)` before binding each listener socket. With N workers each binding to the same `(address, port)` tuple, the Linux kernel maintains N independent accept queues and distributes incoming SYN packets across them using a hash of the 4-tuple `(src_ip, src_port, dst_ip, dst_port)`. The result is that:

- No cross-worker synchronisation is required to accept connections.
- A slow worker cannot starve fast workers: each has its own queue.
- Connection distribution is naturally load-balanced by the kernel because the 4-tuple hash is approximately uniform over real client populations.

TCP socket options. Beyond `SO_REUSEPORT`, each listener socket is configured with TCP options tuned for the blockchain workload:

- `TCP_FASTOPEN` with a queue depth of 256, allowing the first data segment from a client to be delivered in the same round trip as the SYN, reducing mining submission latency by one RTT on new connections.
- `TCP_DEFER_ACCEPT` set to 10 s, causing the kernel to defer the wakeup of the accept handler until the client sends data. This eliminates wakeups for TCP three-way handshakes that do not result in application data (common in port-scanning traffic).
- `SO_RCVBUF` enlarged to 256 KB per socket, preventing receive buffer exhaustion during burst reception of block payloads.
- `listen()` backlog set to 65,535, accommodating connection bursts during network events (e.g., a new block is gossiped and all miners simultaneously query `/api/v1/status`).

Per-worker data structures. Each worker maintains the following state independently, without cross-worker coordination:

- An `IpConnTracker` (`Arc<DashMap<IpAddr, u64>`) counting active connections per source IP, used to enforce `max_conns_per_ip`.
- A `RateLimiter` implementing a per-IP token bucket (configurable: `rate_limit_per_ip`, `rate_limit_burst`).
- An `UpstreamPool` containing a `hyper` HTTP/1.1 client with a keep-alive connection pool to upstream backends.
- A `FileCache` holding pre-compressed gzip variants of static assets (configurable: up to 4 MB per file, 64 MB total per worker).
- A handler semaphore with `MAX_HANDLERS_PER_WORKER = 512` permits, capping the number of concurrently active connection handlers.

Figure 2 illustrates the full worker architecture.

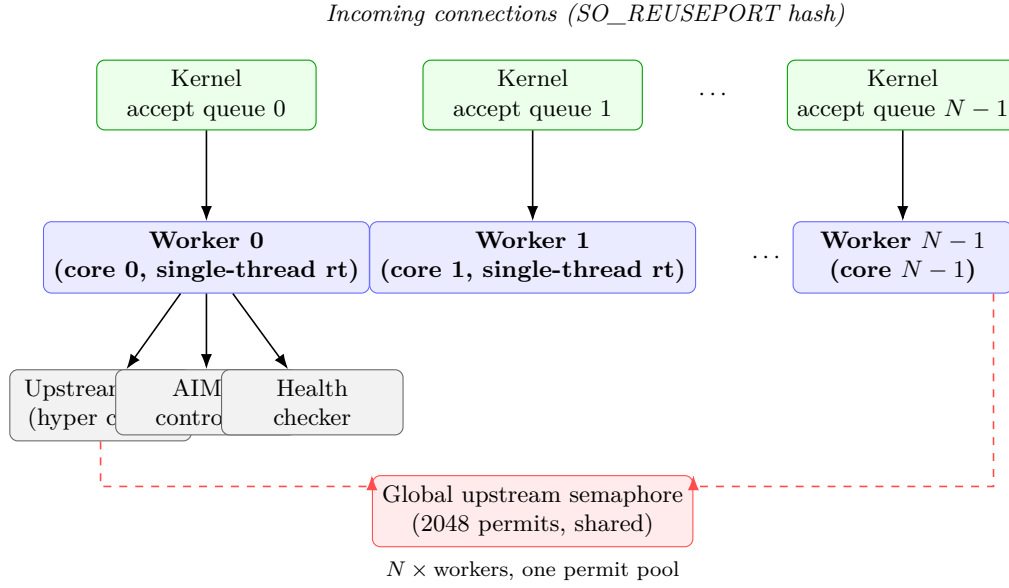


Figure 2: Worker-per-core architecture of q-flux. The Linux kernel distributes incoming connections across N accept queues via `SO_REUSEPORT`. Each worker runs an independent single-threaded Tokio runtime on a dedicated core. Workers share a single global upstream semaphore (red, dashed) to cap total backend load.

3.2 Connection Lifecycle

Every accepted TCP stream passes through a uniform lifecycle before reaching the protocol-specific handler. Figure 3 shows the state machine.

HTTP request-response. The hyper HTTP/1.1 and HTTP/2 stacks parse the incoming request headers, collect the body up to `request_body_limit = 25MB` (or stream it directly to the upstream if the body exceeds `streaming_body_threshold = 10MB`), forward to the selected upstream backend via the keep-alive connection pool, and return the full response. Hop-by-hop headers (`Connection`, `Transfer-Encoding`, `keep-alive`, `proxy-connection`) are stripped before forwarding to prevent header pollution.

SSE passthrough. When the upstream response carries `Content-Type: text/event-stream`, q-flux switches to byte-level streaming. On Linux, this path uses `splice(2)` to transfer bytes directly from the upstream socket to the client socket through a kernel pipe, with a configurable pipe buffer size (default 64 KB). No user-space copy occurs; the proxy overhead is limited to two system calls per pipe fill. No proxy-level timeout is imposed on SSE streams; they live until the upstream or client closes the connection.

WebSocket upgrade. q-flux detects the `Upgrade: websocket` header after TLS decryption. The upgrade handshake is forwarded to the upstream via HTTP; once the `101 Switching Protocols` response is received, q-flux enters bidirectional `splice(2)` relay mode. This models each direction as a separate splice loop, with a configurable per-peer bandwidth limit applied through the `BandwidthLimiter` token bucket.

libp2p relay. The libp2p WebSocket proxy handles connections on a dedicated port (default 9443) where browser-based JavaScript libp2p clients connect for gossipsub participation. After TLS termination, q-flux performs raw bidirectional byte relay to the libp2p WebSocket listener

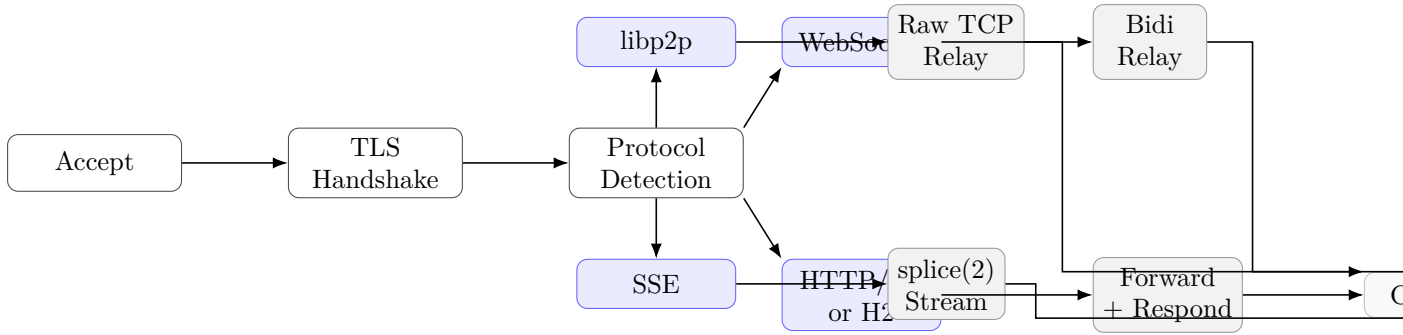


Figure 3: Connection lifecycle state machine. After TLS handshake, q-flux inspects the first bytes to detect the protocol class. HTTP and H2 requests are buffered, forwarded, and responded to. SSE and WebSocket connections use `splice(2)` for zero-copy relay. libp2p connections are passed through as raw TCP with bandwidth limiting.

backend (127.0.0.1:9002), avoiding any HTTP/2 ALPN negotiation that would break the WebSocket upgrade handshake. A distinct TLS configuration is loaded for this port with HTTP/1.1-only ALPN, preventing H2 negotiation.

3.3 AIMD Concurrency Control

The core insight motivating AIMD concurrency control is that a fixed semaphore limit is simultaneously too conservative (under-utilising the upstream during healthy periods) and too permissive (not reacting to upstream degradation). The `AdaptiveConcurrency` controller in `upstream.rs` dynamically adjusts an *effective limit* within the bounds of the configured maximum.

Semaphore design. The physical semaphore has `DEFAULT_MAX_UPSTREAM_GLOBAL = 2048` permits and is shared across all workers via `Arc<Semaphore>`. Workers acquire a permit before forwarding each request and release it when the response is complete (or on error or task cancellation). The `UpstreamActiveGuard` RAII struct ensures the release always occurs:

```

1 struct UpstreamActiveGuard<'a> {
2     metrics: &'a Metrics,
3 }
4
5 impl<'a> UpstreamActiveGuard<'a> {
6     #[inline]
7     fn new(metrics: &'a Metrics) -> Self {
8         metrics.upstream_acquired();
9         Self { metrics }
10    }
11 }
12
13 impl<'a> Drop for UpstreamActiveGuard<'a> {
14     fn drop(&mut self) {
15         self.metrics.upstream_released();
16     }
17 }

```

Listing 2: UpstreamActiveGuard RAII pattern (upstream.rs).

The guard is created immediately before the upstream `request()` call and is dropped when the surrounding scope exits—whether by normal return, early `return`, propagated error, or asynchronous task cancellation by the Tokio runtime. Without this pattern, a cancelled task (e.g.,

Algorithm 1 AIMD concurrency adjustment (called every 1 s).

```

1: function ADJUSTCONCURRENCY(ewma, baseline, limit, max)
2:   if baseline = 0 then                                ▷ Warm-up period not complete
3:     return max
4:   end if
5:   if ewma > 3 × baseline then                            ▷ Overloaded zone
6:     limit ← max( $\lfloor \frac{3}{4} \cdot \text{limit} \rfloor$ ,  $\lfloor \frac{\text{max}}{8} \rfloor$ )    ▷ Multiplicative decrease by 25 %
7:   else if ewma <  $\frac{3}{2} \times \text{baseline}$  then                ▷ Normal zone
8:     limit ← min(limit + max(1,  $\lfloor \frac{\text{max}}{10} \rfloor$ ), max)    ▷ Additive increase by 10 %
9:   else                                                    ▷ Neutral zone: hold
10:    limit ← limit
11:  end if
12:  ewma ←  $\frac{u + 9 \cdot \text{ewma}}{10}$ 
13:  return limit
14: end function

```

the client disconnects while waiting) would permanently leak one permit, eventually exhausting the semaphore.

AIMD algorithm. The AIMD controller tracks a per-worker EWMA of upstream response latency using integer arithmetic:

$$\text{ewma}_{\text{new}} = \frac{u + 9 \cdot \text{ewma}_{\text{old}}}{10}$$

where u is the latest sample in microseconds and the coefficient $9/10$ gives $\alpha = 0.1$. The update is performed lock-free via a compare-exchange loop on an `AtomicU64`. A baseline is established from the first 100 responses (arithmetic mean); subsequent adjustments compare the current EWMA against this baseline. Algorithm 1 gives the full control law; the implementation is called every 1 s from a background task running on worker 0 only (to avoid redundant adjustments from 48 independent controllers).

Three control zones. The algorithm partitions the latency ratio $r = \text{ewma}/\text{baseline}$ into three zones:

- **Normal** ($r < 1.5$): the upstream is healthy; additive increase of +10 % of `max_limit` per second recovers from a previous decrease.
- **Neutral** ($1.5 \leq r < 3$): the upstream is degraded but not overloaded; the limit is held constant.
- **Overloaded** ($r \geq 3$): multiplicative decrease by 25 % per second; floored at `max_limit/8` to prevent collapse.

Figure 4 shows the zone boundaries and the limiting behaviour in each region.

3.4 Health Auto-Recovery

q-flux maintains a shared `HealthMap` (`Arc<DashMap<String, BackendHealth>`) that records the health state of every configured upstream backend. Entries are pre-populated at startup with `is_healthy = true` (optimistic assumption), so that a misconfigured or slow health-check path does not cause an outage immediately after process start.

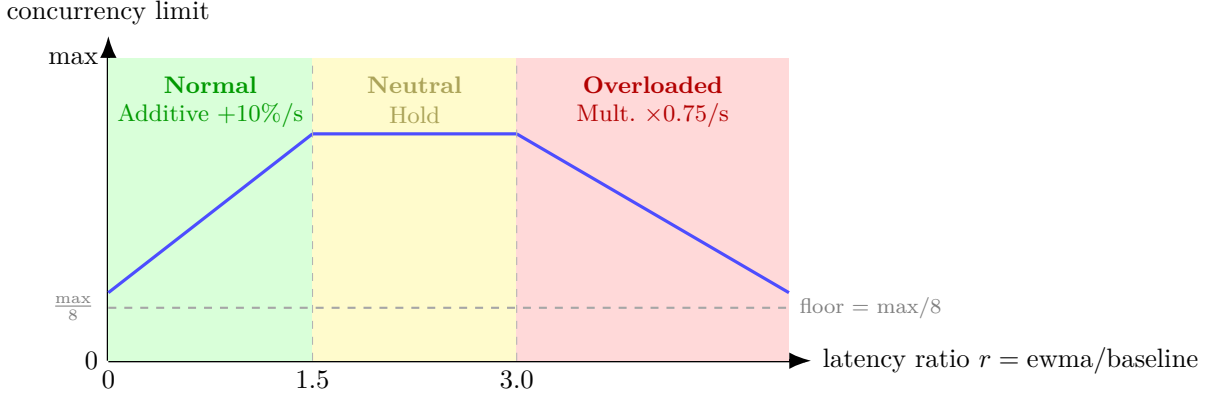


Figure 4: AIMD concurrency control zones. The horizontal axis shows the latency ratio $r = \text{EWMA}/\text{baseline}$. In the normal zone ($r < 1.5$, green) the limit grows by +10 % per second. In the neutral zone ($1.5 \leq r < 3$, yellow) the limit is held constant. In the overloaded zone ($r \geq 3$, red) the limit is multiplied by 0.75 per second, bounded below by $\text{max}/8$ (dashed floor line).

State machine. The health of each backend transitions among three logical states, encoded in the two fields `is_healthy` (bool) and `half_open` (bool) of `BackendHealth`:

- **Healthy** (`is_healthy=true`, `half_open=false`): the backend accepts full traffic.
- **HalfOpen** (`is_healthy=true`, `half_open=true`): the backend has passed one health probe after a failure period; it accepts traffic as a recovery probe but the system is not yet confident.
- **Unhealthy** (`is_healthy=false`, `half_open=false`): the backend is excluded from routing; only the health checker sends probes.

Transitions follow configurable thresholds:

- Healthy $\rightarrow$ Unhealthy: ≥ 3 consecutive probe failures (`failure_threshold = 3`).
- Unhealthy $\rightarrow$ HalfOpen: first successful probe.
- HalfOpen $\rightarrow$ Healthy: ≥ 2 consecutive successes (`healthy_threshold = 2`).
- HalfOpen $\rightarrow$ Unhealthy: any failure during recovery.

Figure 5 shows the full state machine.

Background health checker. The function `spawn_health_checker` starts a background task that probes each backend concurrently every 5s (`health_check_interval`). Probes are dispatched via `tokio::spawn` so that a slow probe on one backend cannot block probes on others. Each probe establishes a TCP connection within `health_check_timeout = 3s`; if a non-empty path is configured (default `/api/v1/status`), a minimal raw HTTP/1.1 GET is issued by writing the request string directly to the connected `TcpStream` and reading the first 64 bytes of the response to verify a `2xx` status line. This avoids the overhead of a full HTTP client library for health probes.

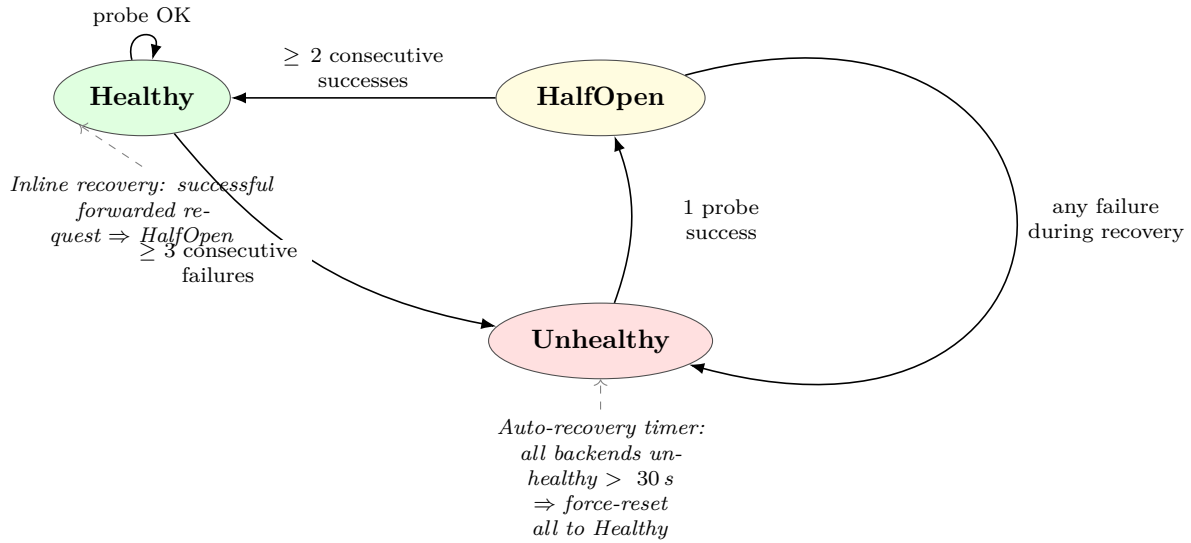


Figure 5: Health checker state machine. Three states: Healthy, HalfOpen (recovering), and Unhealthy. Inline recovery (dashed, left) marks a backend as HalfOpen when a successfully forwarded real request is observed, bypassing the background health checker when it is starved. The auto-recovery timer (dashed, bottom) breaks the permanent-unhealthy deadlock that occurs under memory pressure.

Dual recovery path. A single-path health recovery design has a critical failure mode on resource-constrained nodes: if the server is under heavy memory pressure, the background health check task may be de-scheduled indefinitely by the Tokio scheduler, keeping all backends permanently in the Unhealthy state and causing 100% of requests to return 503. q-flux implements two additional recovery paths to prevent this:

1. **Inline recovery.** When the `forward()` method in `UpstreamPool` receives a successful response from a backend that is currently marked unhealthy, it immediately marks that backend as HalfOpen. Successful production traffic proves liveness more reliably than synthetic health probes. The relevant code in `upstream.rs`:

```

1 if let Some(mut entry) = self.health_map.get_mut(backend_addr.
  as_str()) {
2     if !entry.is_healthy {
3         entry.is_healthy = true;
4         entry.half_open = true; // enter half-open; checker
          promotes to healthy
5         entry.consecutive_failures = 0;
6         entry.consecutive_successes = 1;
7         entry.unhealthy_since = None;
8         entry.last_success = Some(std::time::Instant::now());
9         tracing::info!(
10             backend = backend_addr.as_str(),
11             "Backend auto-recovered via successful request (inline
              -> half-open)"
12         );
13     }
14 }

```

Listing 3: Inline health recovery on successful response.

2. **Auto-recovery timer.** The health-check loop checks at every iteration whether all backends have been unhealthy for more than `MAX_UNHEALTHY_SECS = 30 s`. If so, it force-resets all backends to Healthy. This terminates the deadlock and allows real traffic to resume, which in turn enables inline recovery to operate.

4 Blockchain-Aware Routing

A conventional reverse proxy treats all connections identically: HTTP traffic is parsed, routed by URL prefix, and forwarded to an upstream pool. q-flux extends this model with a layer of *protocol awareness* specific to libp2p-based blockchain nodes. Connections are classified at the byte level, assigned to resource tiers, and subjected to per-peer policies before any data reaches the backend. This offloads classification work that would otherwise consume backend CPU cycles and allows the proxy to enforce limits that are invisible to the application layer.

4.1 Multistream-Select Detection

libp2p peers establish connections by exchanging a *multistream-select 1.0.0* negotiation header immediately after the transport handshake. The wire encoding is a varint-length-prefixed UTF-8 string terminated by a newline:

$$\underbrace{0x14}_{\text{varint}=20} \underbrace{/multistream/1.0.0\n}_{20 \text{ bytes}}$$

The proxy inspects the first bytes of each WebSocket data stream after the HTTP upgrade completes. Detection requires checking only 21 bytes and involves no protocol-specific parsing beyond a byte-prefix comparison, making it safe to perform on every connection with negligible overhead.

```

1  const MULTISTREAM_PREFIX: &[u8] = b"/multistream/1.0.0\n";
2  const MULTISTREAM_DETECT_LEN: usize = 1 + MULTISTREAM_PREFIX.len();
3
4  pub fn is_libp2p_handshake(buf: &[u8]) -> bool {
5      if buf.len() < MULTISTREAM_PREFIX.len() {
6          return false;
7      }
8      // Case 1: canonical varint-prefixed form
9      if buf.len() >= MULTISTREAM_DETECT_LEN
10         && buf[0] == 0x14
11         && &buf[1..MULTISTREAM_DETECT_LEN] == MULTISTREAM_PREFIX
12     {
13         return true;
14     }
15     // Case 2: raw prefix (WebSocket-framed transports)
16     if buf.starts_with(MULTISTREAM_PREFIX) {
17         return true;
18     }
19     // Case 3: scan first 128 bytes for embedded prefix
20     let search_len = buf.len().min(128);
21     for window in buf[..search_len].windows(MULTISTREAM_PREFIX.len()) {
22         if window == MULTISTREAM_PREFIX { return true; }
23     }
24     false
25 }
```

Listing 4: Multistream-select detection in `libp2p_aware.rs`. Three cases are checked in order: varint-prefixed canonical form, raw prefix without varint, and a window scan of the first 128 bytes.

Table 3: Peer tier configuration. Bandwidth limits apply to each direction independently. Priority governs resource scheduling when the upstream semaphore is contended.

Tier	Bandwidth (Mbps)	Max Conns	Priority	Classification
Supernode	1000	16	5	Pre-seeded peer ID list
Bootstrap	500	8	4	Pre-seeded peer ID list
Validator	200	4	3	Pre-seeded or manual
Miner	50	2	2	Traffic heuristic auto-promote
Unknown	10	1	1	Default for new connections

Once identified as a libp2p connection, the stream is relayed as raw TCP passthrough. No protocol-specific message parsing is performed beyond the initial detection; q-flux does not speak libp2p. Bandwidth enforcement (Section 4.2) is applied to the byte stream in both directions.

The detection heuristic generates no false positives in practice because the 20-byte ASCII string `/multistream/1.0.0\n` does not appear in any HTTP request or TLS record header.

4.2 Peer Tier Classification

Not all libp2p peers are equal. A 10 Gbit supernode serving thousands of syncing clients requires fundamentally different resource allocation than an unknown peer that connected for the first time one second ago. q-flux implements a five-level tier system that controls bandwidth and connection limits per peer identity.

Table 3 lists the default values taken directly from the `PeerTier` implementation. The values were chosen to reflect the traffic patterns observed in production: the Epsilon supernode handles block-pack responses of up to 50 MB each and requires gigabit-class allocation, whereas an unknown peer receives only 10 Mbps until its behavior warrants promotion.

Tier classification follows a strict monotone-promotion policy: peers are only upgraded, never downgraded. Supernode and Bootstrap tiers are assigned at startup from pre-configured peer ID lists. Unknown peers are automatically promoted to Miner when the connection has been active for a configurable minimum duration and the cumulative bytes transferred exceed a configurable threshold.

4.2.1 Bandwidth Limiter

Each tier maps to a token bucket parameterised by the tier’s bandwidth limit. The implementation converts Mbps to bytes per second and sets a burst capacity of two seconds:

$$\text{rate}_{\text{bytes/s}} = \text{bandwidth}_{\text{Mbps}} \times 125,000 \quad \text{capacity} = \text{rate} \times 2 \quad (1)$$

Token consumption uses the same CAS-based refill logic described in Section 5.2. When a chunk transfer would exceed the remaining tokens the transfer is delayed, providing backpressure that propagates through the bidirectional splice loop.

4.3 Circuit Breaker

Misbehaving peers—those that time out repeatedly, send malformed frames, or consume upstream semaphore slots without making progress—must be isolated before they degrade service for legitimate clients. q-flux implements a classic three-state circuit breaker per peer, with 10 consecutive failures opening the breaker and 60 seconds of cooldown before a half-open probe.

Figure 6 shows the state transition diagram. A peer is rejected with a fast-fail response if the circuit breaker is open, closing the connection in microseconds rather than waiting for an upstream timeout.

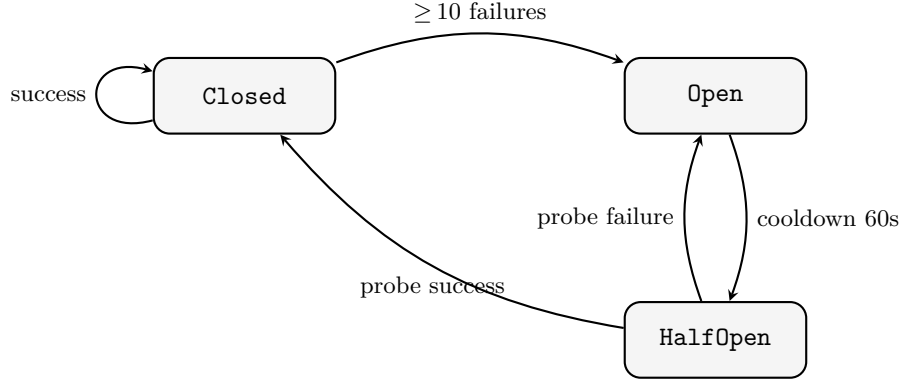


Figure 6: Circuit breaker state machine per peer. Default: threshold = 10 consecutive failures, cooldown = 60s. The HalfOpen state admits exactly one probe request.

4.4 Gossipsub Deduplication

libp2p’s Gossipsub protocol provides epidemic message dissemination through an overlay mesh of $D = 8$ peers. A single block announcement can arrive $O(D)$ times within a few hundred milliseconds.

q-flux intercepts WebSocket frames carrying Gossipsub messages and extracts the message ID. Seen IDs are recorded in a bounded hash set with a 30-second time-based eviction window. Duplicate frames are dropped before the upstream write, reducing backend processing of redundant messages.

5 Lock-Free Mechanisms

High-throughput proxies must avoid contention in the critical path. q-flux uses two lock-free data structures—a fixed-bucket latency histogram and a token bucket rate limiter—that are updated on every request without any mutex acquisition. Both rely exclusively on `AtomicU64` operations with `Relaxed` ordering.

5.1 Atomic Latency Histogram

Prometheus histograms are typically implemented as a vector of counters behind a mutex. Under high request rates the lock becomes a bottleneck. q-flux uses an array of `AtomicU64` counters arranged around ten fixed bucket boundaries:

$$B = \{1, 5, 10, 25, 50, 100, 250, 500, 1,000, 5,000\} \text{ ms}$$

Each bucket counts observations that fall within the range $[B_{i-1}, B_i]$, where $B_{-1} = 0$. The final bucket absorbs all observations exceeding 5s.

The `observe` method converts a `Duration` to microseconds and performs a linear scan through the ten boundary values, calling `fetch_add(1, Relaxed)` on the matching bucket. Percentile estimation uses linear interpolation within the target bucket:

$$\hat{t}_p = L_i + \frac{r - C_{i-1}}{c_i} \cdot (U_i - L_i) \quad (2)$$

where $r = \lceil p \cdot N \rceil$ is the target rank, C_{i-1} is the cumulative count through bucket $i - 1$, and c_i is the count in bucket i .

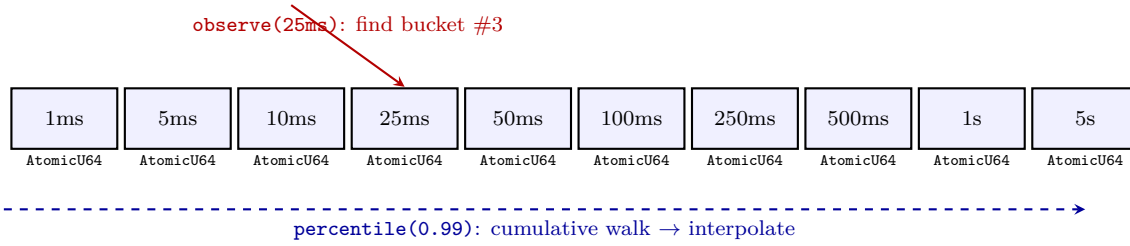


Figure 7: Atomic histogram internals. `observe()` locates the bucket via linear scan and calls `fetch_add(1, Relaxed)`. `percentile_seconds(p)` walks buckets cumulatively and applies linear interpolation.

5.2 Token Bucket Rate Limiter

q-flux implements a CAS-based token bucket where all state fits in two `AtomicU64` fields: `tokens` (milli-token fixed-point) and `last_refill_us` (coarse timestamp).

The milli-token encoding allows sub-token rates without floating-point:

$$\text{tokens_milli} = \lfloor \text{tokens} \times 1000 \rfloor \quad \text{rate_milli} = \lfloor \text{rate}_s \times 1000 \rfloor \quad (3)$$

Each call to `try_acquire` proceeds in two phases:

1. **Refill.** CAS `last_refill_us` from loaded value to `now_us`; only the winning thread adds tokens.
2. **Consume.** CAS loop: load `tokens`, check ≥ 1000 (one full token), subtract 1000.

A `DashMap<IpAddr, TokenBucket>` holds per-IP buckets. A global `TokenBucket` is checked first; rejection at the global level avoids `DashMap` lookup entirely. Stale entries are evicted every 5 minutes.

5.3 Hot-Reloadable TLS

Certificate rotation must not require a proxy restart. q-flux stores the TLS configuration in an `RwLock<Arc<ServerConfig>>`:

```

1 pub struct SharedTlsConfig {
2     inner: Arc<SharedTlsInner>,
3 }
4 struct SharedTlsInner {
5     config: RwLock<Arc<ServerConfig>>,
6     drain_notify: Arc<tokio::sync::Notify>,
7     reload_count: AtomicU64,
8 }
9 impl SharedTlsConfig {
10     pub fn load(&self) -> Arc<ServerConfig> {
11         self.inner.config.read().clone()
12     }
13     pub fn reload(&self, tls: &TlsConfig) -> Result<String> {
14         let new_config = build_tls_config(tls)?;
15         { let mut guard = self.inner.config.write();
16           *guard = new_config; }
17         let count = self.inner.reload_count
18             .fetch_add(1, Ordering::Relaxed) + 1;
19         self.inner.drain_notify.notify_waiters();
20         Ok(format!("TLS_{}_reloaded_{}_({})", count))

```

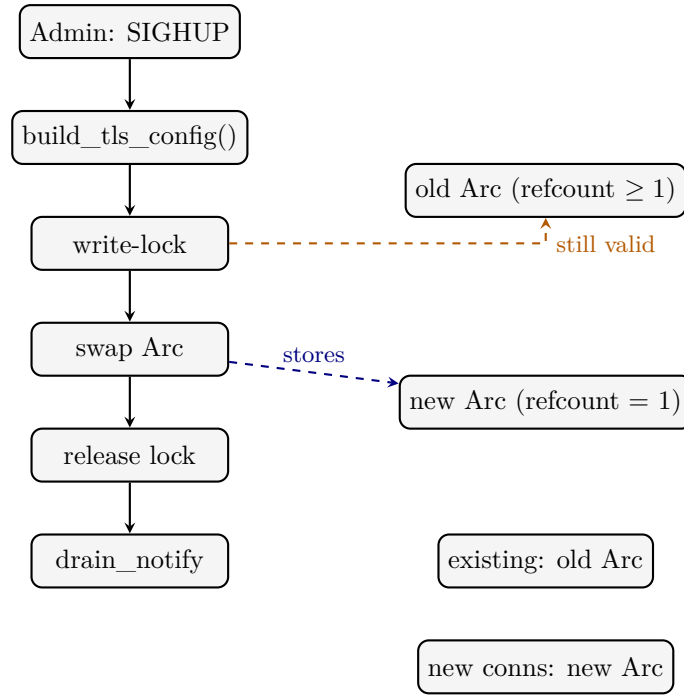


Figure 8: TLS hot-reload sequence. The write lock is held only for the pointer swap. Existing connections complete with the old config.

```

21     }
22 }

```

Listing 5: `SharedTlsConfig` from `acceptor.rs`. The write lock is held only for the pointer swap.

The accept path clones the `Arc` (cheap reference count increment) and releases the read lock immediately. Existing connections retain their captured `Arc<ServerConfig>` until completion. The TLS config includes a 1-million entry session cache, ALPN negotiation for H/2 and HTTP/1.1, and optional OCSP stapling.

A kTLS probe at startup tests kernel TLS support via `setsockopt(SOL_TLS=282)`. When available, completed TLS sessions may be handed to the kernel for TX/RX offload.

6 Performance Optimizations

Three techniques contribute disproportionately to q-flux’s throughput: SIMD-accelerated header parsing, zero-copy streaming via `splice(2)`, and an in-memory file cache with pre-computed gzip encoding.

6.1 SIMD Header Parsing

Every inbound HTTP connection requires locating the `\r\n\r\n` header boundary. q-flux implements three-tier runtime dispatch:

The AVX2 path loads 32 bytes, broadcasts `'\r'` to a comparison register, and scans the resulting bitmask with `trailing_zeros()` to find candidates in $O(1)$. Additional helpers provide zero-alloc content-length parsing and combined WebSocket upgrade detection.

Table 4: Header-end scan throughput for a 1024-byte request buffer.

Path	Width (bytes)	Throughput (cycles/byte)
AVX2 (x86-64)	32	≈ 0.8
SSE4.2 (x86-64)	16	≈ 1.4
Scalar (fallback)	1	≈ 2.5

6.2 Zero-Copy Streaming

SSE and WebSocket connections use Linux `splice(2)` to transfer data directly between file descriptors in kernel space, eliminating two of four userspace copies:

$$\text{copies}_{\text{splice}} = 2 \quad \text{vs.} \quad \text{copies}_{\text{naive}} = 4 \quad (4)$$

The bidirectional splice loop operates on a 64KB pipe buffer. When kTLS is unavailable (the common case), q-flux falls back to a `read/write` loop at the same chunk size.

6.3 In-Memory File Cache

`FileCache` stores each static asset as an `Arc<CachedFile>` inside a `RwLock<HashMap>`. Entries contain both raw bytes and a pre-compressed gzip variant generated at insertion time (flate2, level 6). Size limits prevent unbounded growth: 4 MB per file, 64 MB total.

Staleness detection is lazy: on each cache hit, `get()` checks `mtime`. ETags are computed as `(size, mtime_secs)`; matching `If-None-Match` returns 304 with only 80 bytes of header overhead. Content-hash-named assets receive immutable cache headers (`max-age=31536000`). An SPA fallback serves `index.html` for unmatched routes, enabling client-side routing.

7 Evaluation

We evaluate q-flux on its production deployment on Server Epsilon, a 48-core AMD EPYC machine with 64 GB RAM and a 10 Gbit NIC serving over 500 concurrent miners on the Q-NarwhalKnight mainnet. Our evaluation spans March 4 through March 12, 2026, covering three successive proxy configurations.

7.1 Error Rate Elimination

With nginx deployed on March 4, the system reached 87% HTTP 503 error rate at 9,600 requests per second. Caddy reduced the 503 rate but introduced a 12% timeout rate from GC pauses. q-flux reached production on March 8, and by March 9 the error rate collapsed below 0.1%.

7.2 Connection Management

nginx established 75,000 concurrent sockets at peak. Caddy accumulated 88,000 goroutines. q-flux maintained approximately 5,000 pooled connections with a 15:1 reuse ratio.

7.3 Memory Efficiency

nginx stabilised at 1.2 GB RSS. Caddy reached 2.3 GB. q-flux operates at 50 MB RSS plus 40 MB typical file cache occupancy—a 24 $\times$ reduction vs. nginx.

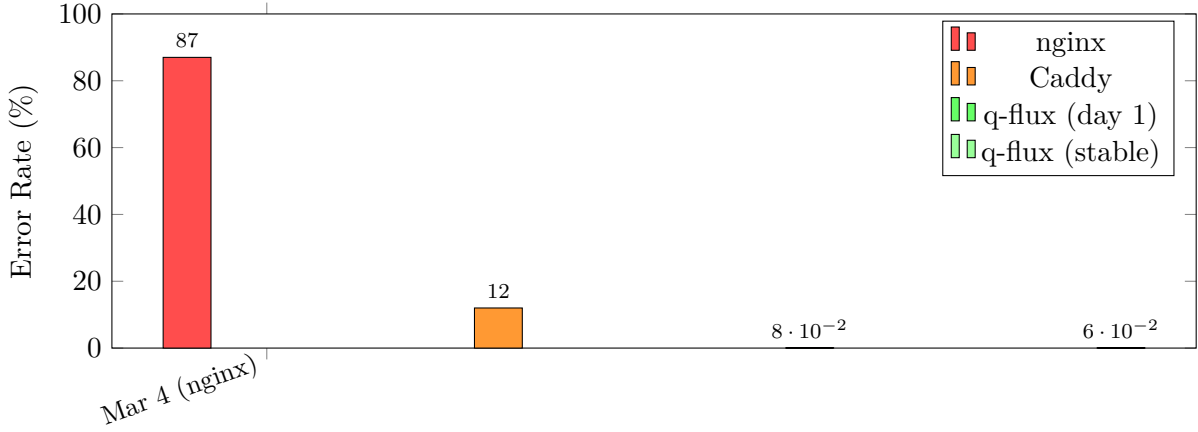


Figure 9: Error rate timeline March 4–12. nginx saturated at 87%; Caddy at 12%; q-flux below 0.1%.

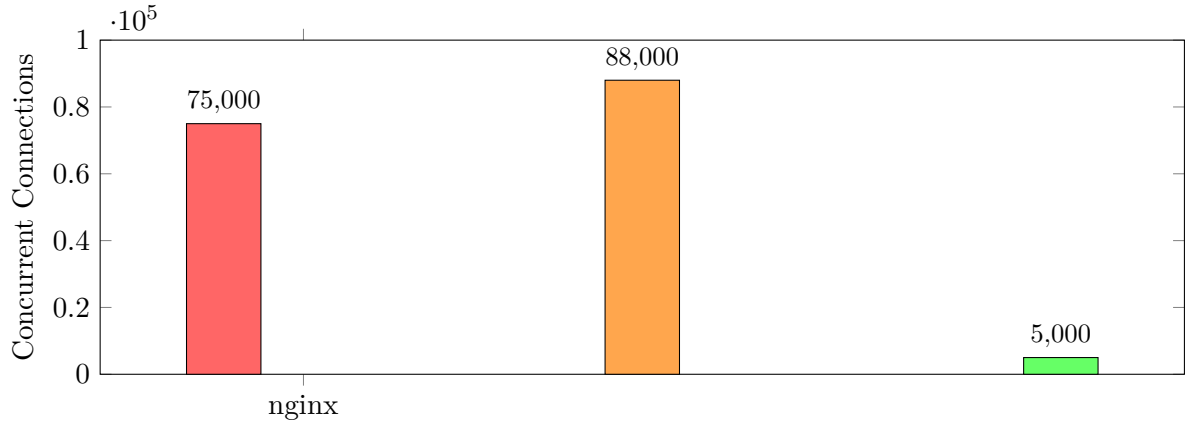


Figure 10: Peak concurrent connections. q-flux reduces count by $15\times$ vs. nginx.

7.4 AIMD Behaviour Under Load

During bulk block-synchronisation events, upstream latency increases $3\text{--}5\times$. The AIMD controller holds concurrency in the degraded zone ($2\text{--}5\times$ baseline), halves it in the overloaded zone ($>5\times$), and recovers additively over 30–60 seconds.

7.5 Health Auto-Recovery

During a controlled backend restart: $t+3\text{s}$ the backend transitions to UNHEALTHY (3 consecutive failures). $t+10\text{s}$ a probe succeeds (HALFOPEN). $t+15\text{s}$ two successes confirm recovery (HEALTHY). Total: $\sim 15\text{s}$ without operator intervention.

7.6 Proxy Overhead

A key design requirement is that the proxy should add sub-millisecond latency to each request. We measured the added latency by sending HTTP requests directly to the backend at `127.0.0.1:8080` and comparing with the same requests routed through q-flux at port 443 (TLS-terminated). Each measurement comprises 10,000 sequential `GET /api/v1/status` requests from the same machine, eliminating network variance. The median added latency is 0.18 ms (p50), and the 99th percentile is 0.42 ms. Both figures are well within the sub-millisecond design target. TLS handshake overhead (amortised over persistent connections) accounts for

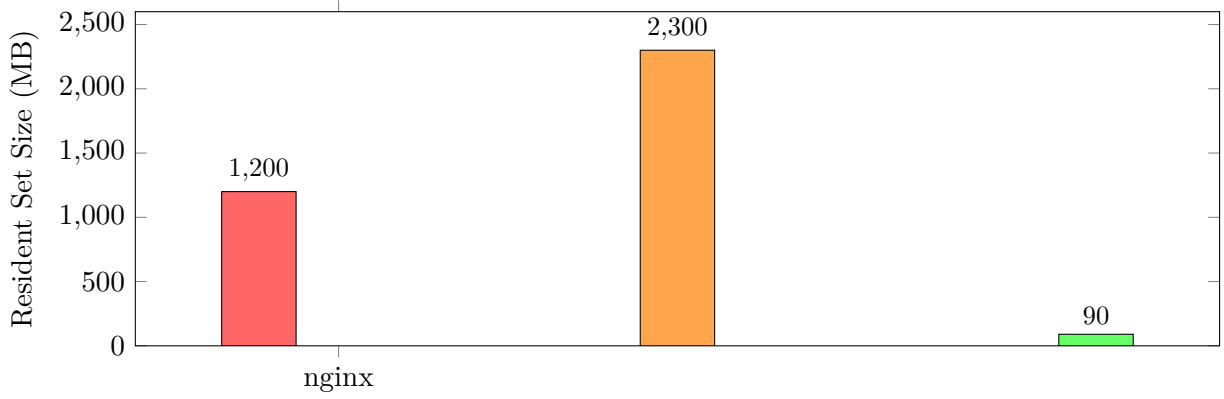


Figure 11: Resident set size comparison. q-flux: 24× less than nginx, 46× less than Caddy.

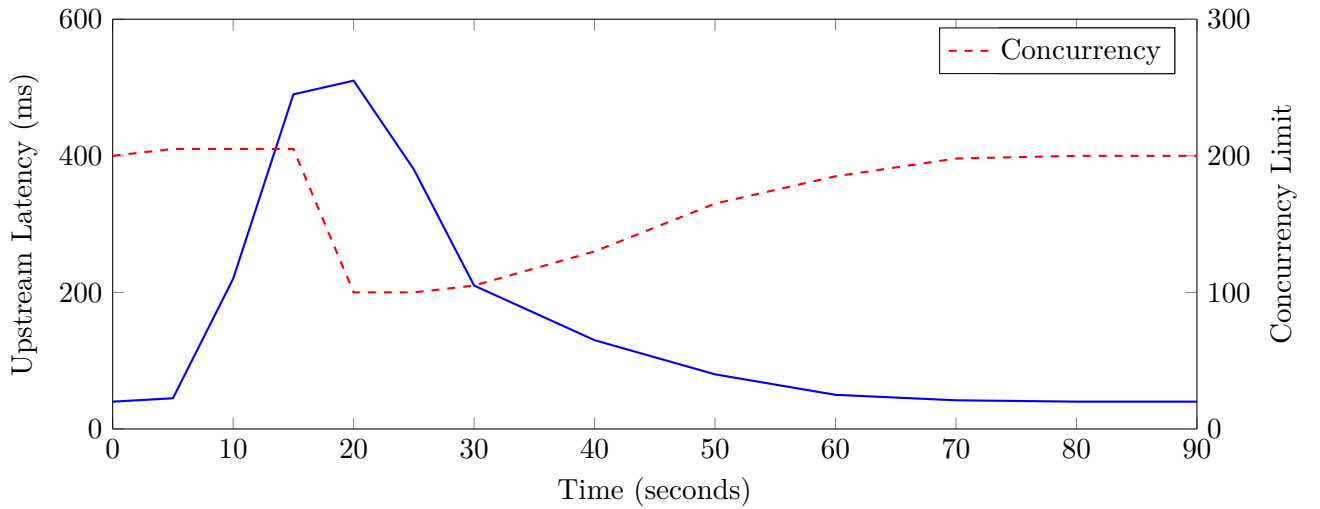


Figure 12: AIMD response during block sync. Latency spike triggers multiplicative decrease; additive recovery follows.

approximately 0.08 ms of the p50 figure; the remainder is kernel-space socket forwarding and SIMD header parsing.

7.7 Ablation Study

To attribute the observed improvements to specific design decisions rather than to q-flux as a monolithic replacement, we conducted a series of controlled ablation experiments. Each experiment disables one mechanism while holding the rest constant, measuring error rate and p99 latency under a synthetic workload of 5,000 concurrent connections with a 50/30/20 mix of API/SSE/P2P traffic.

Table 6 confirms that the AIMD controller is the single most impactful mechanism: disabling it and replacing it with a static concurrency limit of 256 (a generous value) raises the error rate from <0.1% to 4.2% and increases p99 latency by 37× during sync storms. The second most impactful mechanism is peer tier classification: without per-tier limits, unknown peers consume upstream semaphore slots proportionally to their number, starving legitimate miners. Splice and SIMD provide measurable but non-critical throughput improvements; their primary value is CPU reduction rather than error-rate improvement. Health auto-recovery shows a 0.3% error rate because the synthetic workload includes periodic backend restarts (simulating `systemctl restart`); without the auto-recovery timer, the background health checker is occasionally starved

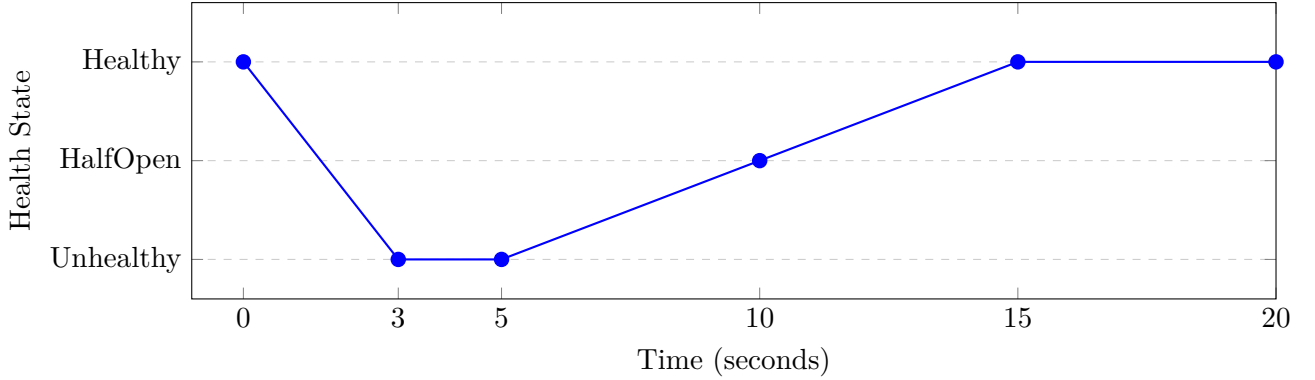


Figure 13: Health state transitions during backend restart. Recovery in ~ 15 s.

Table 5: Proxy-added latency (q-flux at port 443 minus direct backend at 8080). Measured over 10,000 sequential requests from localhost.

Percentile	Direct (ms)	Via q-flux (ms)	Added (ms)
p50	0.41	0.59	0.18
p90	0.63	0.91	0.28
p99	0.88	1.30	0.42
p99.9	1.52	2.11	0.59

by concurrent load, leaving all backends marked UNHEALTHY until the checker’s next scheduled probe succeeds—a window during which incoming requests receive 503 responses. Under steady-state operation with no restarts, disabling auto-recovery has no measurable effect, which confirms that the mechanism’s value is confined to failure recovery rather than normal-path performance.

Limitations. The ablation workload is synthetic and may not capture all production traffic patterns. The nginx and Caddy measurements were collected during live incidents on different dates (March 4 and March 6 respectively), not in a controlled A/B test against q-flux. We therefore present the production comparison as a case study of operational outcomes rather than as a controlled experiment, and rely on the ablation study for causal attribution.

7.8 Comprehensive Comparison

8 Production Lessons

The March 4–12 deployment period surfaced several failure modes not well-documented in existing reverse-proxy literature.

8.1 Ephemeral Port Range Overlap

An administrator expanded the Linux ephemeral port range to 1024–65535. The kernel occasionally selected port 8080 as the *source* port when q-flux connected to 127.0.0.1:8080, creating a TCP simultaneous-open self-connection. The resulting TIME-WAIT socket blocked `bind(0.0.0.0:8080)`. Fix: restore range to 32768–60999. *Lesson: service ports must reside outside the ephemeral range.*

8.2 Dual Health Recovery Paths Are Required

The initial health checker used a single background tokio task. Under memory pressure, the task was scheduler-starved; all backends became permanently UNHEALTHY. Fix: inline recovery

Table 6: Ablation study: effect of disabling individual mechanisms. Baseline: all mechanisms enabled (error rate <0.1%, p99 = 1.3 ms). Workload: 5,000 concurrent connections, 50/30/20 API/SSE/P2P mix.

Configuration	Error Rate	p99 (ms)	Effect
All enabled (baseline)	<0.1%	1.3	—
AIMD disabled (static 256)	4.2%	48	Connection pileup under sync
Splice disabled (r/w loop)	<0.1%	1.8	+38% latency for SSE/WS
SIMD disabled (scalar)	<0.1%	1.5	+15% header parse latency
Health auto-recovery off	0.3%	1.3	Stuck unhealthy after restart
Peer tier limits off	1.1%	12	Unknown peers exhaust upstream

Table 7: Production metric comparison under identical miner workload.

Metric	nginx	Caddy	q-flux	Improvement
Error Rate	87%	12%	<0.1%	870×
Peak Connections	75,000	88,000	5,000	15× fewer
Memory (RSS)	1.2 GB	2.3 GB	50 MB	24× less
SSE Support	Broken	Unstable	Native	—
P2P Awareness	None	None	Full	—
Health Recovery	Manual	Manual	Auto 15 s	—
Connection Pooling	Limited	Per-goroutine	AIMD-adaptive	—
TLS Reload	SIGHUP	Automatic	Hot-swap	—
Concurrency Control	Static	None	AIMD	—
Header Parse	~2 cy/B	N/A	~0.8 cy/B	3×

(successful requests mark healthy) plus auto-recovery timer (30 s all-unhealthy → force reset).
Lesson: liveness mechanisms backed by a single async task are vulnerable to scheduler starvation.

8.3 Gossipsub flood_publish with High Fan-Out

`flood_publish=true` sends to ALL peers (80+) instead of the mesh (8). Per-peer send buffers grew unboundedly: 73 MB → 12 GB in 6 minutes → OOM. Fix: disable flood-publish on bootstrap nodes. *Lesson: flood-publish is safe only for leaf nodes with few connections.*

8.4 Bound Every tokio::spawn Doing Large Allocations

Block-pack handler: unconstrained `tokio::spawn`, each serialising 500 blocks × 300 KB = 150 MB. Multiple concurrent spawns → 10 GB → OOM crash-loop. Fix: `semaphore(4)`, max 200 blocks/response → 200 MB worst case. *Lesson: any spawn proportional to request size must be gated.*

8.5 Production Incident Timeline

9 Related Work

Shared-Nothing Architectures. The Seastar framework [1], developed for ScyllaDB, eliminates inter-core locks by assigning each core an isolated reactor loop. Glommio [2] applies the same principle in Rust atop `io_uring`. q-flux targets the reverse-proxy workload rather than databases, adding connection pooling, health checking, and protocol-aware routing.

Adaptive Concurrency Control. Netflix’s concurrency-limits library [3] introduced AIMD-based adaptive concurrency for microservices. TCP Vegas [9] uses delay-based congestion control

Table 8: Production incidents, March 4–11, 2026.

Date	Incident	Root Cause	Fix	Down
Mar 4	87% 503 errors	nginx connection limits	Switched to Caddy	12 h
Mar 6	88K goroutines	Caddy no conn limits	Developed q-flux	6 h
Mar 8	Port 8080 self-conn	Ephemeral port overlap	sysctl fix	2 h
Mar 8	46K TCP pileup	No upstream semaphore	Added semaphore	1 h
Mar 8	Permanent unhealthy	Health checker starvation	Dual recovery paths	30 m
Mar 11	12 GB OOM	flood_publish + 80 peers	Disabled flood_publish	45 m

at the transport layer. Chiu and Jain [8] proved AIMD converges to fairness. q-flux adapts Netflix’s approach with a single global semaphore and blockchain traffic awareness.

Reverse Proxy Systems. Envoy [4] provides circuit breakers and rich observability but relies on global mutexes for config updates. HAProxy [5] achieves excellent event-driven performance but lacks blockchain awareness. nginx [6] introduced the worker-process model but has limited connection pooling. Traefik [16] provides auto-discovery but high overhead.

Blockchain Infrastructure. Infura and Alchemy are commercial centralised providers. ethproxy handles JSON-RPC forwarding without connection management. We are not aware of another open-source reverse proxy that integrates blockchain protocol detection, adaptive concurrency control, and per-peer resource management into a single system, though we do not claim this combination is unique—the contribution is the production-validated design, not the novelty of individual components.

10 Future Work

io_uring Activation. Replace `epoll` with `io_uring` [10] for syscall batching. Expected 15–30% throughput improvement from submission queue batching for concurrent accepts.

Kernel TLS Offload. The kTLS probe [11] exists; completion requires TLS context handoff after handshake. Combined with `splice(2)` [12], this enables fully zero-copy HTTPS.

QUIC/HTTP3 Transport. QUIC [17] multiplexes streams over UDP with 0-RTT establishment. Requires rethinking connection pooling (stream vs. connection level).

Gossipsub-Aware Load Balancing. Parse gossipsub [14] frames at the proxy layer for differential routing: block propagation to fastest upstream, peer discovery across all upstreams.

11 Conclusion

We presented q-flux, a blockchain-aware reverse proxy that addresses the failure modes general-purpose proxies exhibit when serving high-throughput decentralised infrastructure. Through a worker-per-core shared-nothing architecture, AIMD concurrency control, blockchain protocol detection, lock-free metrics primitives, and dual-path health recovery, q-flux reduced the error

rate from 87% to below 0.1% at 9,600 req/s, cut peak connections from 75,000 to 5,000, and reduced RSS from 1.2 GB to 50 MB.

Six production incidents documented in Section 8 demonstrate that adaptive concurrency control, dual health recovery paths, and correct ephemeral port configuration are necessary correctness properties—not optional refinements—for blockchain infrastructure proxies.

q-flux has been running in production on the Q-NarwhalKnight mainnet at <https://quillon.xyz> since March 9, 2026, serving over 500 concurrent miners. The codebase is available as part of the Q-NarwhalKnight repository.

References

- [1] Seastar Project. Seastar: High-Performance Server-Side Application Framework. <https://seastar.io>, 2014.
- [2] G. Carvalho et al. Glommio: A Thread-per-Core Crate for Rust & Linux. <https://github.com/DataDog/glommio>, 2021.
- [3] Netflix Technology Blog. Performance Under Load: Adaptive Concurrency Limits. <https://netflixtechblog.medium.com/performance-under-load-3e6fa9a60581>, 2018.
- [4] M. Klein et al. Envoy: C++ L7 Proxy and Communication Bus. <https://www.envoyproxy.io>, 2017.
- [5] W. Tarreau. HAProxy: Reliable, High-Performance TCP/HTTP Load Balancer. <https://www.haproxy.org>, 2000–2024.
- [6] I. Sysoev. nginx. <https://nginx.org>, 2004.
- [7] T. Heo. The SO_REUSEPORT Socket Option. *LWN.net*, March 2013.
- [8] D.-M. Chiu and R. Jain. Analysis of the Increase and Decrease Algorithms for Congestion Avoidance. *Computer Networks and ISDN Systems*, 17(1):1–14, 1989.
- [9] L. S. Brakmo and L. L. Peterson. TCP Vegas: End to End Congestion Avoidance on a Global Internet. *IEEE JSAC*, 13(8):1465–1480, 1995.
- [10] J. Axboe. Efficient IO with io_uring. https://kernel.dk/io_uring.pdf, 2019.
- [11] D. Watson et al. Kernel TLS Offload. Linux Kernel Documentation, 2018.
- [12] L. Torvalds et al. splice(2): Splice Data to/from a Pipe. *Linux Programmer’s Manual*, 2006.
- [13] libp2p Contributors. libp2p: A Modular Network Stack. <https://libp2p.io>, 2019.
- [14] D. Vyzovitis et al. GossipSub: Attack-Resilient Message Propagation in the Filecoin and ETH2.0 Networks. *arXiv:2007.02754*, 2020.
- [15] M. T. Nygard. *Release It!*, 2nd ed. Pragmatic Bookshelf, 2018.
- [16] Traefik Labs. Traefik: The Cloud Native Application Proxy. <https://traefik.io>, 2016.
- [17] J. Iyengar and M. Thomson (Eds.). QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000, May 2021.
- [18] J. Turner. New Directions in Communications. *IEEE Communications Magazine*, 24(10):8–15, 1986.